MEMORIA DE LA PRÁCTICA:

Aplicación web para el acceso de pacientes

Escenarios

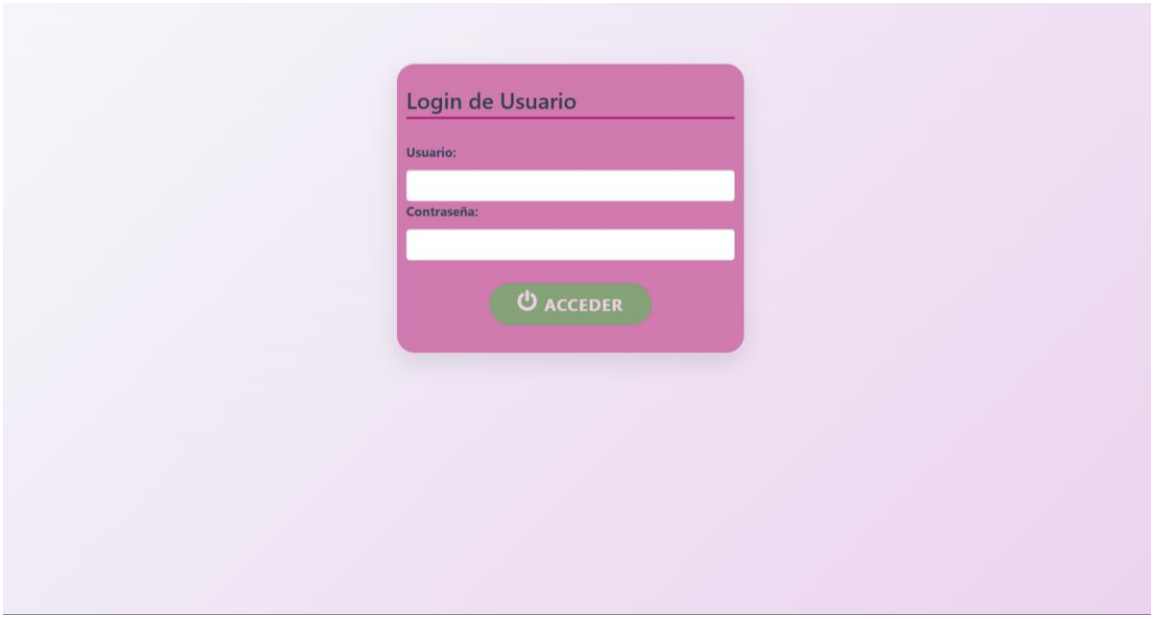


Figura 1. Escenario de Inicio de sesión.

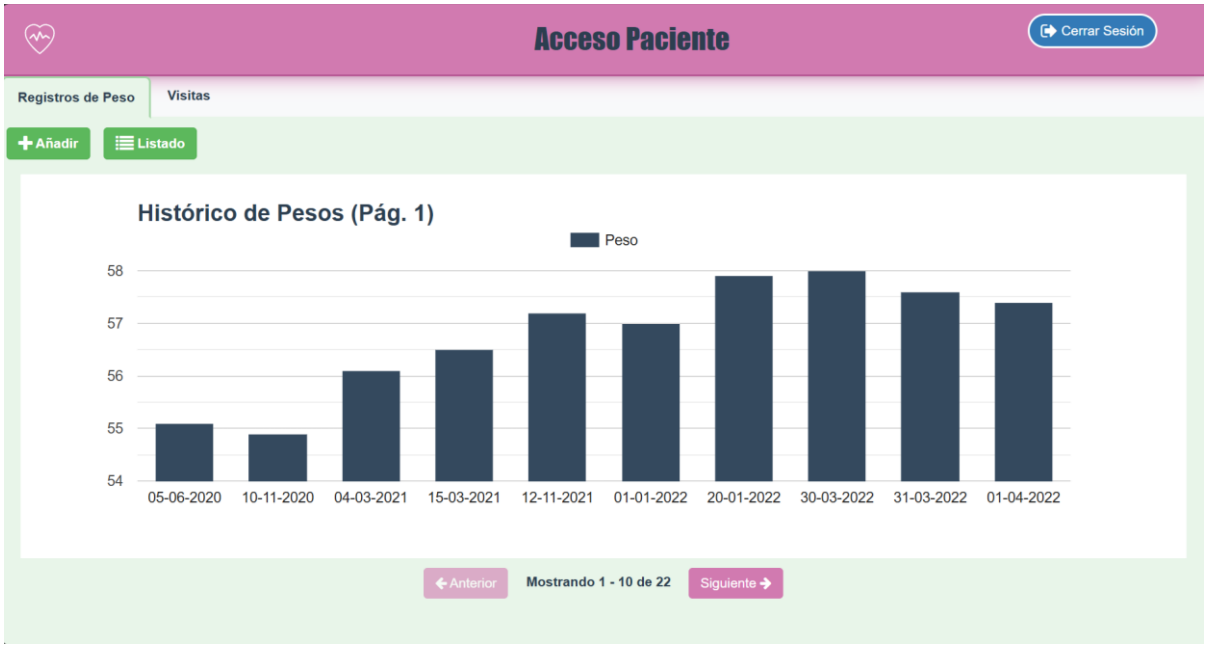


Figura 2. Escenario de Registros de Peso.

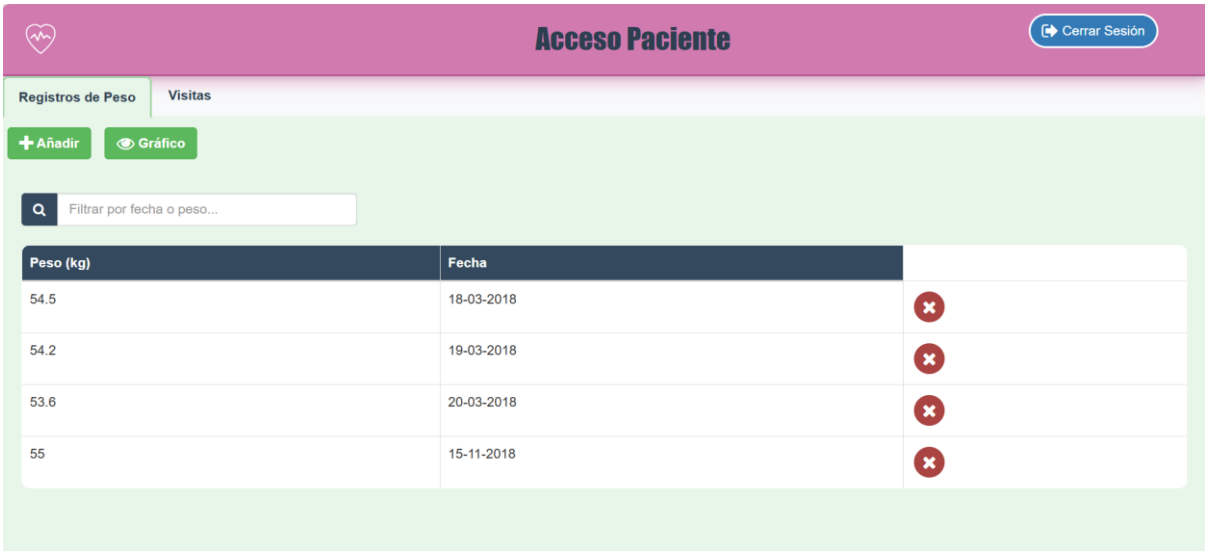


Figura 3. Listado de registros de peso de un paciente.



Figura 4. Listado de visitas.



Figura 5. Formulario de ingreso de nuevos registros de peso.

Explicación del código

Apartado 4: formulario de ingresos de registros nuevos (escenario registros de peso)

Para implementar el escenario de añadir registro de peso se ha seguido la lógica empleada en la aplicación de gestión de pacientes. En **patientData.html**, dentro del * *, se ha escrito el código que se muestra a continuación.

```
<div id="weightForm" ng-show="addWeight==1">
   <form name="pesoForm" novalidate>
      <table>
         <tr>
            <td>
               <div class="form-group">
                  <strong>Peso (kg):</strong>
                  <input type="number" name="Weight" ng-model="WeightData.peso"
class="form-control" placeholder="Ej: 70.5" required />
               </div>
            </td>
            <td>
               <div class="form-group">
                  <strong>Fecha:</strong>
                  <input type="date" name="Date" ng-model="WeightData.fecha"
class="form-control" required />
               </div>
            </td>
         </tr>
      </table>
      <div style="margin-top:20px;">
         <a class="btn btn-success" ng-click="createNewWeight();">
            <i class="fa fa-check-circle fa-lg"></i> Crear
         </a>
         <a class="btn btn-danger" ng-click="cancelNewWeight();">
            <i class="fa fa-times-circle fa-lg"></i> Cancelar
         </a>
      </div>
   </form>
</div>
```

Dentro de este div, se ha definido una estructura de tabla que alberga dos campos de entrada fundamentales: un *input* de tipo numérico para registrar el valor del peso en kilogramos y un *input* de tipo fecha para el calendario, ambos vinculados mediante la directiva *ng-model* a un objeto de datos denominado *WeightData*. Asimismo, se han incluido dos botones de acción que emplean iconos de FontAwesome, asociándolos a través de la directiva *ng-click* a las funciones de creación y cancelación definidas en el controlador.

En el archivo **patientController.js** se define el objeto para recoger los datos, la función para mostrar el formulario y las funciones para *Crear* y *Cancelar*. Al inicio del controlador, en la sección de variables se inicializa el objeto con campos vacíos para el peso y la fecha.

```
$scope.WeightData = {
   peso: "",
   fecha: ""
};
```

La función *showForm* se encarga de modificar la variable de visibilidad para desplegar el formulario cuando el usuario presiona el botón añadir.

```
$scope.showForm = function() {
   $scope.addWeight = 1;
};
```

La función *cancelNewWeight* permite cancelar la operación y *createNewWeight* gestiona la validación y el envío de la información. Dentro de esta última, se ha aplicado una corrección horaria al objeto de fecha mediante el método *setHours* para asegurar que, al convertir el dato al huso horario UTC antes de su inserción en la base de datos, no se produzca un desfase que retrase el registro al día anterior. Una vez validada la presencia de datos, se invoca al servicio correspondiente para realizar la petición al servidor.

```
$scope.cancelNewWeight = function() {
   $scope.addWeight = 0;
   $scope.WeightData = { peso: "", fecha: "" };
};

$scope.createNewWeight = function() {
   if ($scope.WeightData.peso && $scope.WeightData.fecha) {
      $scope.addWeight = 0;

      var date = new Date($scope.WeightData.fecha);
      date.setHours(date.getHours() + 2);

      var newWeightData = {
         PAC_ID: $scope.PatientID,
         PESO: $scope.WeightData.peso,
         FECHA: date
```

```
        };

        Patient.addWeight(newWeightData).success(function(data) {
            $window.location.reload();
        });

        $scope.WeightData = { peso: "", fecha: "" };
    } else {
        alert("Para guardar un nuevo peso debes rellenar todos los campos");
    }
};
```

El servicio **patient.js** se ha actualizado para incluir el método *addWeight*, el cual utiliza el servicio *$http* para realizar una petición de tipo POST hacia la ruta de la API, enviando el objeto con el identificador del paciente, el valor del peso y la fecha procesada.

```
patientFactory.addWeight = function(weightData) {
    return $http.post('/api/weights', weightData);
};
```

Finalmente, en el archivo **api.js** del servidor, se ha configurado la ruta receptora que ejecuta una consulta SQL específica. Siguiendo las recomendaciones del enunciado, se ha empleado la sentencia *INSERT INTO ... ON DUPLICATE KEY UPDATE*, lo que permite que la base de datos gestione de forma inteligente la entrada: si ya existe un peso registrado para esa fecha concreta, el valor se actualiza; de lo contrario, se crea un registro nuevo en la tabla *tbpeso*.

```
apiRouter.post('/weights', function(req, res) {
    req.getConnection(function(err, conn) {
        if (err) return res.send(err);

        var query = conn.query("INSERT INTO tbpeso set ? ON DUPLICATE KEY UPDATE
?",
        [req.body, req.body], function(err, rows) {
            if (err) return res.send(err);
            res.json({ message: 'Peso registrado correctamente' });
        });
    });
});
```

Apartado 5: listado de registros existentes (escenario registros de peso)

Para implementar esta funcionalidad se ha utilizado la directiva *ng-repeat* de Angular para recorrer los datos que ya tenemos almacenados en el controlador y mostrarlos en una tabla HTML. Tal y como indica el enunciado, estos datos ya se recuperan al iniciar la sesión para poder dibujar la gráfica, por lo que solo necesitamos vincularlos a la vista. En **patientData.html**, dentro del * *, se ha añadido el código que se muestra a continuación.

```
<div id="weightsList" ng-show="showGraph==0 && addWeight==0">
  <table class="table table-bordered table-striped">
    <thead>
      <tr style="background-color: #f5f5f5;">
        <th>Peso (kg)</th>
        <th>Fecha</th>
      </tr>
    </thead>
    <tbody>
      <tr ng-repeat="weight in weightsList">
        <td>{{weight.PESO}}</td>
        <td>{{weight.FECHA_ALTER}}</td>
      </tr>
    </tbody>
  </table>
</div>
```

En **patientController.js** se ha definido la función *changeView* para que gestione el cambio entre la gráfica y la tabla.

```
$scope.changeView = function(view) {
  $scope.showGraph = view;
};
```

Apartado 6: borrar registros existentes (escenario registros de peso)

En **patientData.html** se ha añadido una nueva columna a la tabla de listado de pesos con el botón de borrado, escribiendo el siguiente código dentro de la directiva *ng-repeat*.

```
  <td>
    <span class="fa-stack fa-lg" ng-click="deleteWeight(weight.PESO_ID, PatientID);"
style="cursor:pointer;">
      <i class="fa fa-circle fa-stack-2x text-danger"></i>
      <i class="fa fa-times fa-stack-1x fa-inverse"></i>
    </span>
  </td>
```

En **patient.js** se ha añadido la función que realiza la petición DELETE enviando ambos parámetros en la URL.

```
patientFactory.deleteWeight = function(weightID, patientID) {
    return $http.delete('/api/deleteWeights/' + weightID + '/' + patientID);
};
```

En **api.js** se ha configurado la ruta para recibir los dos parámetros, ejecutar el borrado y devolver la lista actualizada del paciente.

```
apiRouter.route('/deleteWeights/:weightId/:patientId')
.delete(function(req, res) {
    var weightId = req.params.weightId;
    var patientId = req.params.patientId;
    req.getConnection(function(err, conn) {
        if (err) return res.send(err);
        conn.query("DELETE FROM tbpeso WHERE PESO_ID = ?", [weightId], function(err,
result) {
            if (err) return res.send(err);
            conn.query('SELECT DATE_FORMAT(FECHA, "%d-%m-%Y") FECHA_ALTER,
FECHA, PESO, PESO_ID FROM tbpeso WHERE PAC_ID=? ORDER BY FECHA ASC',
            [patientId], function(err, weights) {
                if (err) return res.send(err);
                res.json(weights);
            });
        });
    });
});
```

En **patientController.js** se ha definido la función que se comunica con el servicio y actualiza la variable *$scope.weightsList*. Además, se ha incluido un aviso de confirmación para la eliminación de datos.

```
$scope.deleteWeight = function(weightID, patientID) {
    if (confirm("¿Estás seguro de que deseas eliminar este registro de peso?")) {
        Patient.deleteWeight(weightID, patientID).success(function(data) {
            $scope.weightsList = data;
            $window.location.reload();
        });
    }
};
```

Apartado 7: escenario de visitas asociadas

En **patient.js** se ha añadido la función para solicitar las visitas asociadas al paciente.

```
patientFactory.associatedVisits = function(id) {
    return $http.get('/api/associatedVisits/' + id);
```

```
};
```

En **api.js** se ha creado la ruta que realiza el JOIN entre las tablas para mostrar el nombre del médico y los detalles de la visita.

```
apiRouter.route('/associatedVisits/:patientId')
.get(function(req, res) {
   var patientId = req.params.patientId;
   req.getConnection(function(err, conn) {
      if (err) return res.send(err);
      var sql = 'SELECT tbvisitas.*, tbmedicos.MED_NOMBRE FROM tbvisitas ' +
            'INNER JOIN tbmedicos ON tbvisitas.MED_ID = tbmedicos.MED_ID ' +
            'WHERE PAC_ID = ? ORDER BY FECHA_VIS ASC';
      conn.query(sql, [patientId], function(err, visits) {
         if (err) return res.send(err);
         res.json(visits);
      });
   });
});
```

En **patientController.js** se ha incluido una llamada para cargar las visitas con la siguiente función, empleando la variable *$scope.PatientID* que ya teníamos inicializada.

```
   $scope.getVisits = function() {
      Patient.associatedVisits($scope.PatientID).success(function(data) {
         $scope.visitsList = data;
      }).error(function(err) {
         console.log("Error al obtener visitas: ", err);
      });
   };
  $scope.getVisits();
```

En **patientData.html**, dentro de la pestaña de Visitas (*uib-tab index="1"*), se ha escrito el código que añade la tabla para mostrar los datos.

```
<uib-tab index="1" heading="Visitas">
   <div id="visitsList" style="margin-top:20px;">
      <table class="table table-bordered table-striped">
         <thead>
            <tr style="background-color: #3b83bd; color: white; font-weight: bold;">
               <td>Médic@</td>
               <td>Fecha de la visita</td>
               <td>Diagnóstico</td>
            </tr>
         </thead>
         <tbody>
```

```
        <tr ng-repeat="visit in visitsList" track by $index" >
          <td>{{visit.MED_NOMBRE}}</td>
          <td>{{visit.FECHA_VIS | date:'dd-MM-yyyy'}}</td>
          <td>{{visit.DIAGNOSTICO || 'Sin especificar'}}</td>
        </tr>
      </tbody>
    </table>
    <div ng-show="visitsList.length == 0" class="alert alert-info">
      No tienes visitas registradas todavía.
    </div>
  </div>
</uib-tab>
```

Mejoras

Mejoras de aspecto

- **Interfaz login**:
  Para la interfaz de inicio de sesión se ha modificado el archivo login.css. Se ha transformado el contenedor principal *#boxLogin* en una tarjeta con bordes redondeados de 20px y una sombra sutil (*box-shadow*) que le otorga profundidad sobre el fondo. Se ha aplicado un degradado lineal (*linear-gradient*) al fondo de la página que transita desde un tono blanquecino a un morado suave, creando una estética limpia y profesional. El título se ha configurado como un bloque independiente para asegurar alineación vertical con los campos de texto y se ha personalizado la tipografía. grafía. El botón *Acceder* cuenta con bordes redondeados y tipografía clara para mejorar la legibilidad, además de efecto sombra y agrandamiento del icono en caso de interacción.

- **Cabecera más estética**:
  La cabecera del escenario se ha rediseñado para ocupar todo el ancho superior, utilizando un esquema de colores coherente con el resto de la aplicación. Se ha modificado el elemento h1 para incluir un sombreado sutil que aporta volumen y una franja inferior que delimita claramente el área de navegación. Además, se ha integrado un logotipo en formato SVG mediante el pseudoelemento *::before*, posicionado a la izquierda del título, lo que dota a la interfaz de una identidad visual propia de una aplicación de salud digital.

- **Colores para cada escenario**:

Se ha implementado una diferenciación cromática para mejorar la experiencia de usuario según la pestaña activa. El escenario de "Registros de Peso" utiliza una paleta de tonos verdes claros (*#e8f5e9*), mientras que el de "Visitas" emplea tonos morados claros (*#f3e5f5*). Para eliminar las discontinuidades visuales, se han ajustado las pestañas activas para que su borde inferior desaparezca y se funda cromáticamente con el fondo del escenario correspondiente. Asimismo, se han definido bordes laterales más oscuros en las pestañas para resaltar cuál es la sección que el paciente está consultando en cada momento.

- **Micro-interacciones**:
  Se han añadido efectos de transición suaves (*transition: all 0.3s ease*) en todos los elementos interactivos para mejorar la respuesta visual. Los botones *Añadir*, *Listado*, *Gráfico*, *Cerrar Sesión*, *Crear*, *Cancelar* y *Borrar* (icono fa-stack con la equis roja) presentan un efecto de agrandamiento (*transform: scale*) y una sombra más pronunciada cuando el usuario sitúa el cursor sobre ellos, facilitando la identificación de las áreas clicables. En el caso del botón de cerrar sesión, se ha incluido además un resplandor blanco perimetral para destacar su función de salida, y en el de borrar un efecto brillo. Finalmente, se ha añadido el efecto de resaltado al pasar el ratón (*hover*) en las filas de la tabla de listado.

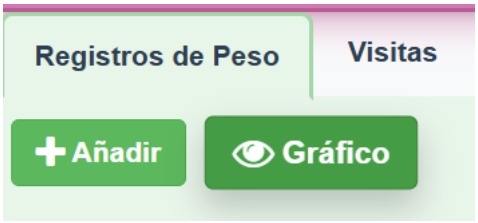


Figura 6. Agrandamiento y efecto sombra en el botón Gráfico.



Figura 7. Agrandamiento y resplandor en el botón Cerrar Sesión.



Figura 8. Agrandamiento y efecto sombra en el botón Cancelar.



Figura 9. Agrandamiento y efecto brillo en el botón Borrar. Sombra sobre la fila seleccionada.

- **Gráfico**:
  En patientController.js, dentro de la sección donde se configura *myCharObject*, se ha actualizado el objeto *options* incluyendo el atributo *series* para cambiar el color de las barras y *titleTextStyle* para el formato del título.

Mejoras de usabilidad

- **Avisos de confirmación**:
  Con el objetivo de evitar la pérdida de datos o salidas involuntarias, se han integrado ventanas de confirmación nativas mediante la función *confirm()* en el archivo patientController.js. Al pulsar el botón de salida, el sistema lanza la pregunta "¿Seguro que desea cerrar sesión?". Solo si el usuario acepta, se procede a eliminar el token *PatientID* del almacenamiento local y se redirige al login. Para el borrado de registros en la función *$scope.deleteWeight*, antes de ejecutar la eliminación definitiva de un registro de peso de la base de datos, se solicita una confirmación.

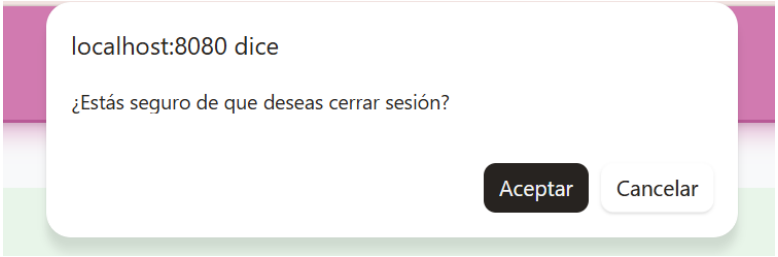


Figura 10. Aviso de confirmación para el cierre de sesión.

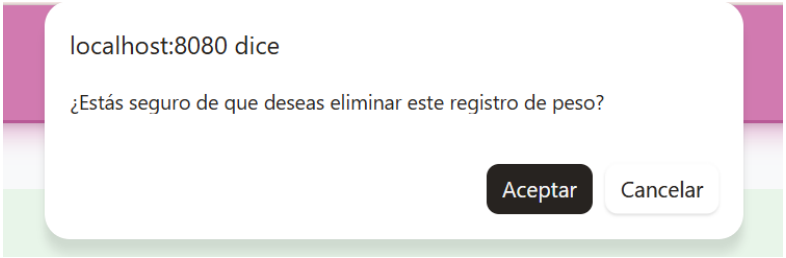


Figura 11. Aviso de confirmación para la eliminación de un registro.

- **Mensajes de éxito tras las operaciones crear y borrar**:

Para confirmar al usuario que sus acciones han tenido éxito sin interrumpir el flujo de navegación con ventanas emergentes, se ha diseñado un sistema de notificaciones dinámicas. Se ha añadido un elemento *alert-success* de Bootstrap en el HTML que se muestra u oculta dinámicamente mediante la directiva *ng-show="successMsg"*. Tras recibir la respuesta positiva del servidor al crear o borrar un peso, se actualiza la variable *$scope.successMsg* con el texto correspondiente. Mediante el servicio *$timeout* de AngularJS, se ha programado la desaparición automática del mensaje tras 3 segundos, manteniendo la interfaz limpia. Se ha sustituido la recarga completa de la página (*reload*) por una función de carga asíncrona (*loadWeights*). Esto permite que la tabla y el gráfico se actualicen instantáneamente mientras el mensaje de éxito es visible, mejorando la percepción de fluidez de la aplicación.

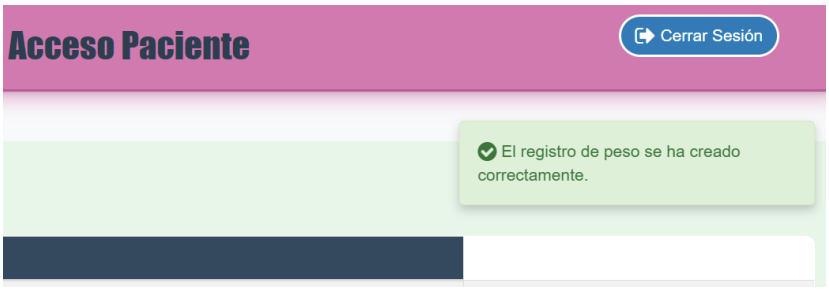


Figura 12. Mensaje de éxito tras la operación crear.

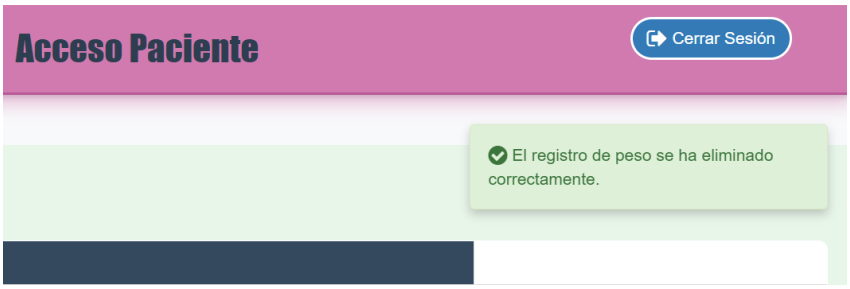


Figura 13. Mensaje de éxito tras la operación borrar.

- **Ordenación cronológica de los registros de peso**:
  Se ha optimizado el procesamiento de datos en el controlador mediante el método *.sort()*. Esta mejora garantiza que, independientemente del orden de inserción de los registros en la base de datos, la aplicación presente la información siguiendo una línea temporal coherente. Al convertir los campos de fecha en objetos *Date* comparables, el sistema reestructura el array de resultados antes de renderizar la tabla de listado y la gráfica, asegurando una visualización lógica de la evolución del peso del paciente.

Nuevas funcionalidades

- **Buscadores**:

Se ha incorporado una herramienta de búsqueda en tiempo real dentro del escenario de "Visitas" modificando el archivo HTML. Esta funcionalidad mejora la usabilidad al permitir la localización rápida de información. Utilizando la directiva *ng-model* vinculada a un filtro en el *ng-repeat* de la tabla, el paciente puede filtrar instantáneamente el histórico por nombre del facultativo, fecha o diagnóstico. Además, se ha añadido un mensaje de alerta que solo aparece si el usuario busca algo que no existe, avisándole de que no hay coincidencias.



Figura 14. Búsqueda en el escenario Visitas.

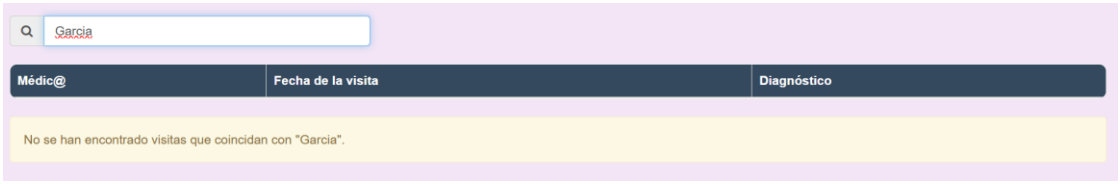


Figura 15. Mensaje de alerta de visita no encontrada.

También se ha implementado un buscador en el escenario de “Registros de pesos”, en concreto en la interfaz de Listado. Esto permite al usuario filtrar los datos almacenados en la tabla por la cifra de peso o la fecha de registro.



Figura 16. Búsqueda en el escenario Registro de pesos.

- **Paginación dinámica en el Gráfico**:
  Con el fin de evitar la saturación visual de la interfaz cuando el historial de registros de peso es muy extenso, se ha desarrollado un sistema de paginación lógica para el componente de visualización gráfica. Esta mejora permite al paciente navegar por su evolución cronológica en bloques controlados, mejorando significativamente la legibilidad de los datos.

Se ha desarrollado una lógica en el controlador que segmenta el array de pesos mediante el método *.slice()*, enviando al gráfico bloques de 10 registros según la variable *currentPage*. En el HTML, se han incorporado botones de navegación con la directiva *ng-disabled* para bloquear el avance en los límites del historial, junto a un indicador dinámico que calcula el rango visible mediante el objeto *Math*. Tras cada operación de crear o borrar, el sistema reordena cronológicamente los datos y reinicia la vista para garantizar la coherencia visual. Finalmente, en el CSS, se han aplicado transiciones y efectos de escalado (*transform: scale*) en las flechas para ofrecer una respuesta táctil clara, junto a una degradación de color en estado deshabilitado que mejora la usabilidad.

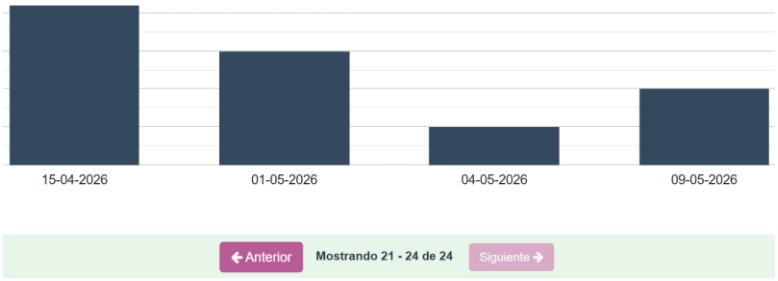


Figura 17. Última página del Gráfico de Registro de pesos.